CIS*3110 Operating Systems

Week 2

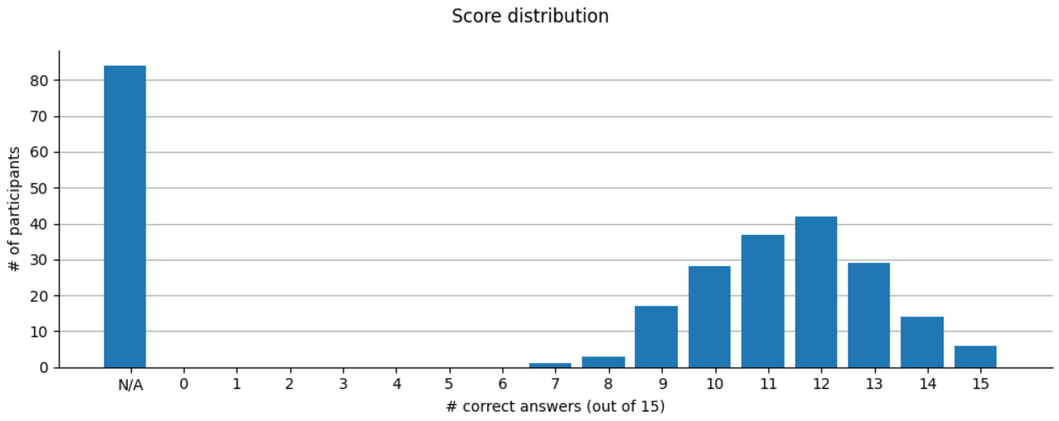
Lecture #3 Virtualization: Processes

Prof. Yulia Kotseruba

January 13, 2026

Quiz 1

- **75%** average (everyone got +1 for the wrong question on the quiz)
- 3 hardest questions:
  - (32%) CPU virtualization allows a) concurrent program execution b) memory sharing c) hard drive sharing d) all of the above
  - (35%) What is the primary function of a CU? a) execute instructions b) data storage c) orchestrate CPU operation d) all of the above
  - (42%) Which of these is part of an OS? Select all that apply a) CPU b) C compiler c) shell d) system call API



Recap

- OS is a **middleware** between user programs and computer hardware
  - Goals: user **convenience**, **efficient use** of hardware, **correctness** of programs
- OS **manages** all aspects of running a program
  - taking user commands via UI, loading program into memory, execution of the program, handling exceptions, managing resources, dealing with I/O, returning the results back to the user
- This is done via **three** general **techniques**
  - **Virtualization** of CPU and memory - sharing compute and memory across processes
  - **Concurrency** - ensuring correctness during parallel execution of processes
  - **Persistence** - managing stored information

What is a process?

- **Process** is an abstraction of a **running program**
- **Process ≠ program**
  - program is **passive** (a set of instructions in a file)
  - process is **active** and has a state that changes over time
  - program becomes a process when its executable is loaded into memory
- Multiple processes may correspond to the same program (e.g. running two copies of the same program)
- Note: processes used to be called **jobs**, both terms mean the same thing

Process components

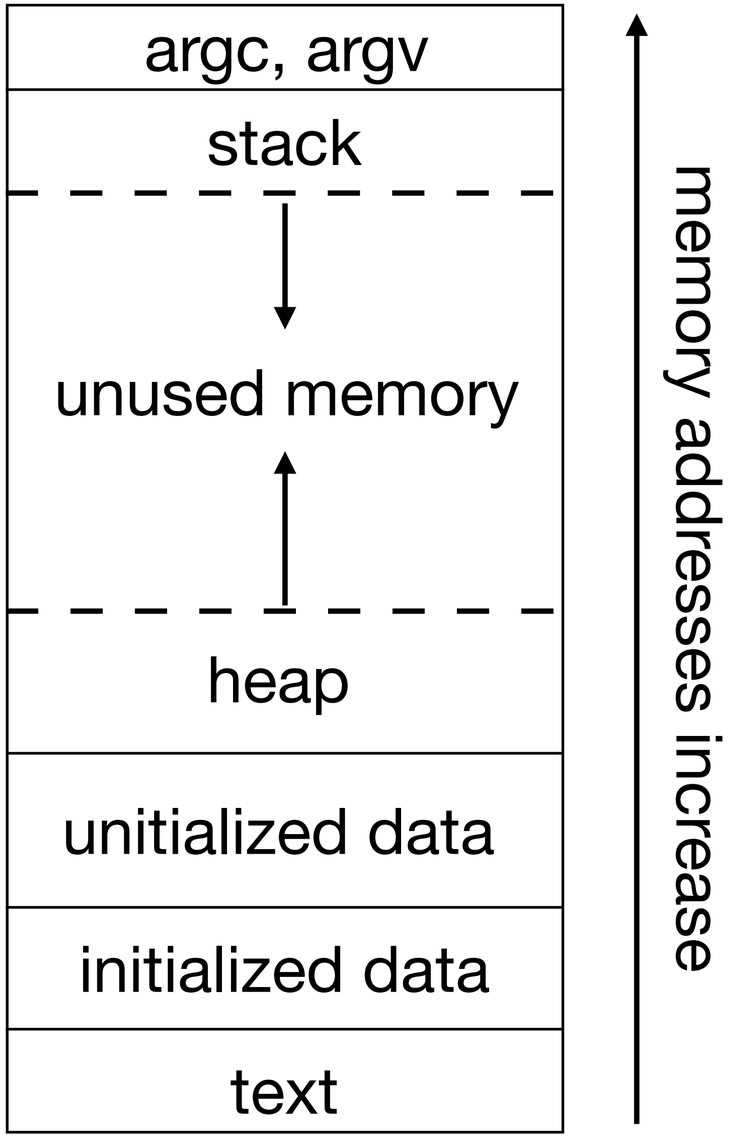
- **Memory** - space allocated for process operation
- **State** - current activity of the process
- **Process data** - a data structure maintained by the OS to keep track of process execution

Process memory

- When a new process is created, a chunk of RAM is allocated for its operation
- Memory of the process is divided into these sections:
  - **Text** - executable code
  - **Data** - global variables
  - **Heap** - dynamically allocated memory
  - **Stack** - temp data storage
    - function params
    - local variables
    - return addresses

Process memory

- **Text** section - executable code
- **Data** section - global variables
- **Heap** section - dynamically allocated memory
- **Stack** section - temp data storage
  - function params
  - local variables
  - return addresses



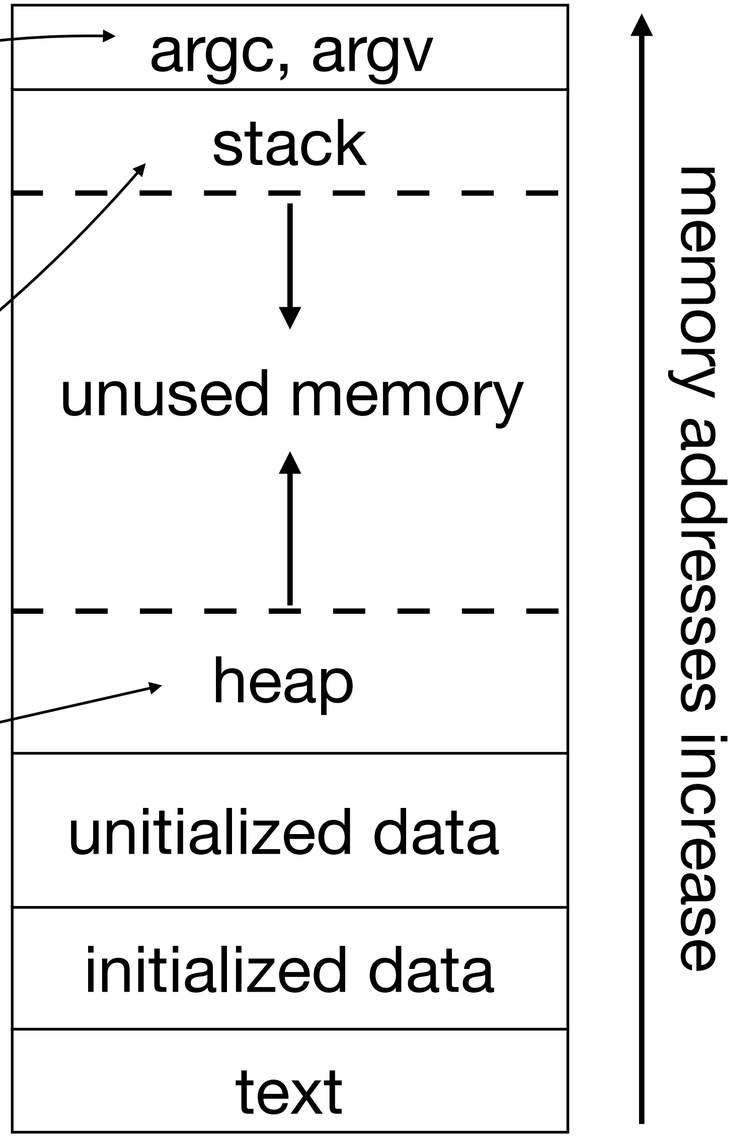
Process memory

```c
#include <stdio.h>
#include <stdlib.h>
int x;
int y = 15;

int main(int argc, char *argv[]){
  int *values;
  int i;

  values = (int *) malloc(sizeof(int)*5);
  for (i = 0; i < 5; i++)
    values[i] = [i];
  return 0;
}
```

mem.c



p.108 from Silberchatz et al 10ed.

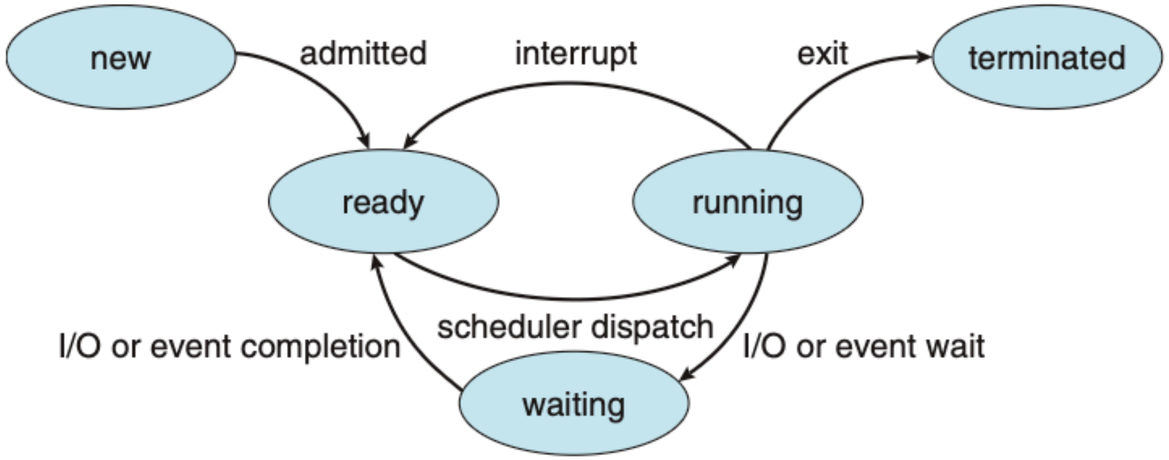
Process state*

- **New** - process is created
- **Running** - executing instructions on a CPU
- **Ready** - waiting to be assigned to a CPU
- **Waiting/Blocked** - waiting for something to happen to continue (e.g. waiting for I/O request to a disk to return)
- **Terminated** - finished execution
- OS decides when and how to change the state of each process

*the names of the states will differ across OSs and textbooks, but the set of states is the same

Process state

- **new -> ready**
  - new process waits for the scheduler
- **ready -> running**
  - CPU begins or resumes the execution
- **running -> terminated**
  - program completes the task, is terminated by its parent, terminated by the OS (e.g. due to invalid instruction, overflow, memory protection violation)
- **running -> ready**
  - scheduler preempted the process to run a different process
- **running -> waiting**
  - process made a system call to wait for I/O operation, resource, sleep, message from another process
- **waiting -> ready**
  - I/O done, process goes to the scheduler's queue



state diagram of process states

Figure 3.2 from Silberchatz et al 10ed.

Process state

- OS mechanisms for tracking and changing process states
  - **Data structures** - process control block (PCB), process list, scheduling queues
  - **System calls** - kernel functions that create, destroy, and manage processes

Process data: PCB

- **Process control block (PCB)** is the representation of the process in an OS
  - **Process state** (new, ready, blocked…)
  - **CPU registers** - program counter, stack pointer, function parameters,…
  - **Address space** - memory layout
  - **CPU scheduling** - process priority, scheduling queues
  - **I/O status** - e.g. a list of open files, any devices that the program interacts with
  - and more

Process data: PCB

- PCB is a process descriptor
- Usually implemented as **C struct(s)**
- What is stored in PCB and how is **OS-dependent**

Process data: Ubuntu PCB

```
struct mm_struct *mm; // memory management info
struct files_struct *files; // open file info
struct thread_struct tss; // task state segment for this stack
                          // (registers, I/O ports, privilege level)
int pid; // process id
volatile long state; // -1 unrunnable, 0 runnable, >0 stopped
long priority; // scheduling priority
unsigned short uid,euid,suid,fsuid;
#ifdef __SMP__
int processor; // which processor runs this
#endif
//links to other related processes (parents, children)
struct task_struct *p_opptr, *p_pptr, *p_cptr, *p_ysptr, *p_osptr;
struct rlimit rlim[RLIM_NLIMITS]; // resource limit/usage
long utime, stime, cutime, cstime, start_time; // timing
```

Process API

- OS provides system calls for process management
  - **Create** - creates a new process (e.g. when you run a program from UI)
  - **Wait** - process waits for something to happen
  - Various **control** functions - calls for suspending/resuming the process
  - **Status** - call for checking on the process (e.g. uptime, state)
  - **Destroy** - may be invoked before or after the program is finished running

Process API: UNIX

- `fork()` - create a new process by duplicating the calling process
- `exec()` - transform currently running process into a different process
- `wait()` - halt execution of a process until a condition is satisfied
- `kill()` - terminate a process

Process API: fork()

```
>>> ./p1
hello world (pid:50600)
hello, I am child (pid:50601)
hello, I am parent of 50601 (pid:50600)
```

the order in which parent and child print is nondeterministic (depends on how they are scheduled)

a process is created by the OS

starts executing from main()

hello world is printed

fork() creates a new (child) process

fork() returns 0 to a child and pid of the child to the parent

child process starts execution from the line fork() is on

```
//includes here
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) { // fork failed, exit
       exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else { // parent goes down this path (original process)
         printf("hello, I am parent of %d (pid:%d)\n", rc, (int) getpid());
    }
    return 0;
}
```

p1.c

Process API: fork()

- **fork()** - creates a **new process** (child) *almost* identical to the calling process (parent)
  - child has a **copy** of the **parent's memory** address space, heap and stack
  - fork() **returns twice**: 0 to a child, child's pid to the parent, -1 if failed
  - child has a **unique PID**, different from the parent
  - child has a **copy** of the parent's **file descriptors**
  - even if the parent process is multithreaded, the child has only **1 thread**
  - concurrency stuff is **not inherited** (locks, signals, alarms)

Process API: wait()

same function as before

but child process now sleeps for 1 sec, parent waits for it to finish and only then returns

```
//includes here
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) { // fork failed, exit
       exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        sleep(1);
   } else { // parent goes down this path (original process)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc: %d) (pid:%d)\n", rc, wc, (int) getpid());

   }
    return 0;
}
```

p2.c

Process API: wait()

```
>>> ./p2
hello world (pid:49997)
hello, I am child (pid:49998)
hello, I am parent of 49998 (wc:49998) (pid:49997)
```

```
//includes here
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) { // fork failed, exit
       exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        sleep(1);
   } else { // parent goes down this path (original process)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc: %d) (pid:%d)\n", rc, wc, (int) getpid());

   }
    return 0;
}
```

hello world is printed

fork() creates a child process that starts executing form this line

child prints its id and sleeps for 1s

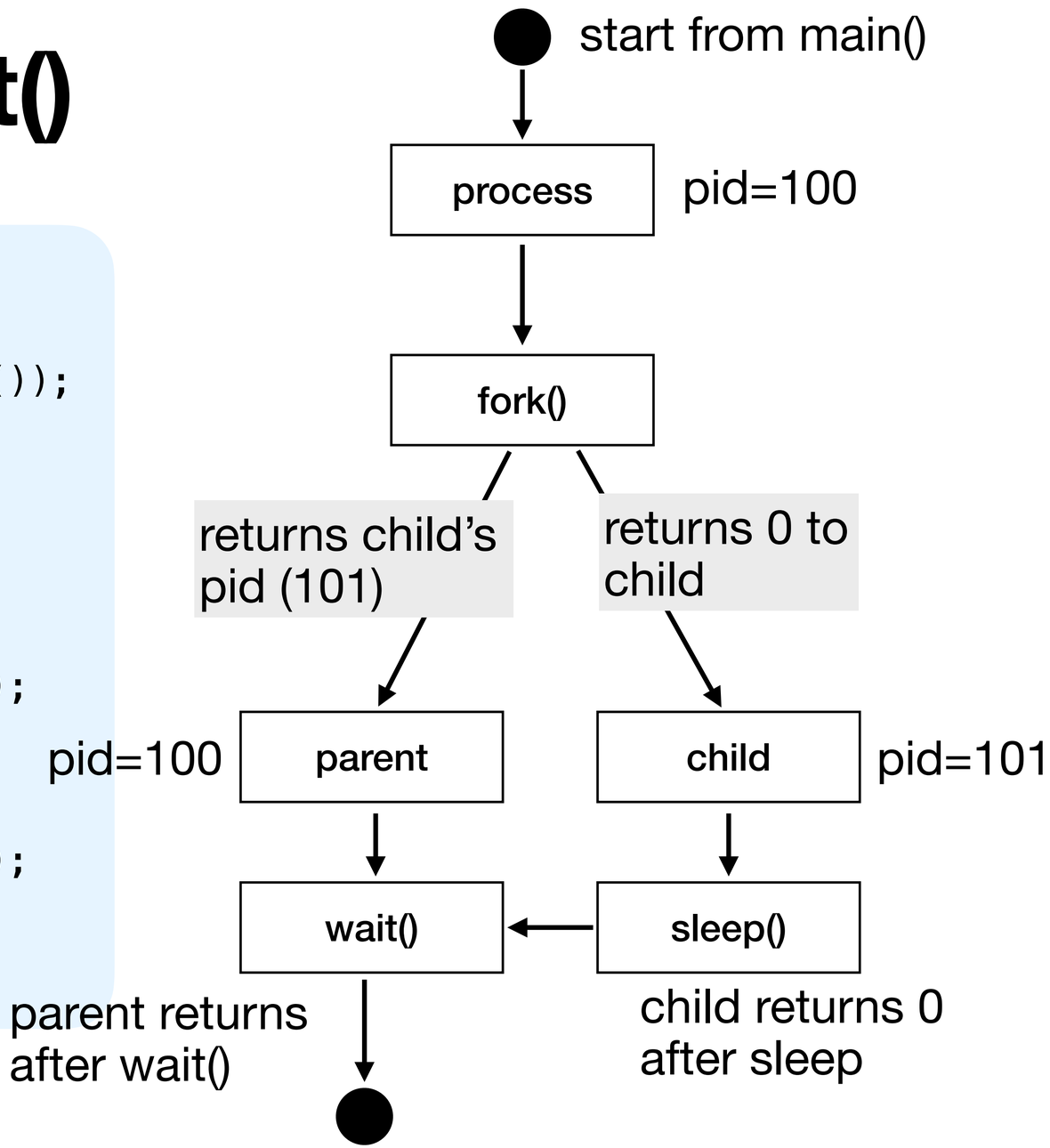
parent waits for child and prints

what difference does wait() make? child will always print first

p2.c

Process API: fork()+wait()

```
int
main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("child (pid:%d)\n", (int) getpid());
       sleep(1);
    } else { //parent
        int wc = wait(NULL);
        printf("parent(pid:%d)\n", (int) getpid());
    }
    return 0;
}
```



Process API: exec()

same function as before

but now child calls exec() which replaces it with another process

```
//includes here
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
     if (rc == 0) { // child (new process)
       printf("hello, I am child (pid:%d)\n", (int) getpid());
      char *myargs[3];
      myargs[0] = strdup("wc");
      myargs[1] = strdup("p3.c");
      myargs[2] = NULL;
      execvp(myargs[0], myargs); // run wc p3.c
       printf("THIS WILL NOT BE PRINTED");
    } else { // parent goes down this path (original process)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc: %d) (pid:%d)\n", rc, wc, (int) getpid());
    }
    return 0; }
```

p3.c

Process API: exec()

```
>>> ./p3
hello world (pid:49999)
hello, I am child (pid:50000)
    32     123     966   p3.c
hello, I am parent of 50000 (wc:50000) (pid:49999)
```

```
//includes here
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
     if (rc == 0) { // child (new process)
       printf("hello, I am child (pid:%d)\n", (int) getpid());
     char *myargs[3];
     myargs[0] = strdup("wc");
     myargs[1] = strdup("p3.c");
     myargs[2] = NULL;
     execvp(myargs[0], myargs); // run wc p3.c
       printf("THIS WILL NOT BE PRINTED");
   } else { // parent goes down this path (original process)
       int wc = wait(NULL);
       printf("hello, I am parent of %d (wc: %d) (pid:%d)\n", rc, wc, (int) getpid());
   }
    return 0; }
```

p3.c

hello world is printed

fork() creates a new process

child prints

execvp() replaces the child with a new process running "wc ./p3.c"

the rest of child code will not be executed

as before, the parent waits for the child, prints and returns

Process API: exec()

- exec() - **does not create a new process**, it **replaces** the program in the current process with another program
  - **stops** the current process
  - replaces the **entire contents** of the current process
    - in our case, it replaces the remainder of p3.c main() function with "wc ./p3.c"
    - **pid of the process stays the same** but memory is reinitialized
  - **loads the new program** into the process space
  - **runs** that program

Process API: exec()

- exec() is a **family of functions**
  - execl, execlp, execle, execv, execvp, execvpe
  - all do the same thing but take different arguments
  - check by running `man 3 exec` in terminal

Process API: exec()

same function as before

but now child closes STDOUT before calling exec()

what difference does this make?

```
//includes here
int main(int argc, char *argv[]) {
    int rc = fork();
     if (rc == 0) { // child (new process)
      close(STDOUT_FILENO);
      open("./output.txt", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
      char *myargs[3];
      myargs[0] = strdup("wc");
      myargs[1] = strdup("p4.c");
      myargs[2] = NULL;
      execvp(myargs[0], myargs);
    } else { // parent goes down this path (original process)
        int wc = wait(NULL);
        assert(wc > 0);
   }
    return 0; }
```

p4.c

Process API: exec()

```
>>> ./p4
>>> cat output.txt
     34    114    884   p4.c
```

```
//includes here
int main(int argc, char *argv[]) {
    int rc = fork();
     if (rc == 0) { // child (new process)
      close(STDOUT_FILENO);
      open("./output.txt", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
      char *myargs[3];
      myargs[0] = strdup("wc");
      myargs[1] = strdup("p4.c");
      myargs[2] = NULL;
      execvp(myargs[0], myargs);
    } else { // parent goes down this path (original process)
        int wc = wait(NULL);
        assert(wc > 0);
   }
    return 0; }
```

fork() creates a new process

child closes stdout

open() will look for the free file descriptor starting with 0

STDOUT_FILENO is 1 and will be the first available

then exec() will change child to run wc p4.c

results will be printed to the file

p4.c

Process API

- Why do we need both fork() and exec()
  - fork() creates a **new process** which runs the **same program** as parent
  - exec() **changes** the child's **program** without affecting the parent
- This is good design because it is modular and separates concerns
  - **fork()** creates **new processes**
  - **exec()** loads **new programs**

Process API

- **`fork()`, `exec()` and `wait()` enable most of the Unix shell functionality**
  - **Example #1**: running `` `echo "hi"` `` in the terminal
    - the shell process calls `fork()` to create a child process
    - the child process calls `exec()` with `` `/bin/echo' `` and `'hi'` as arguments
    - `echo` is then executed instead of the original child program
    - the shell process waits for the child using `wait()`

Process API

- **`fork()`, `exec()` and `wait()` enable most of the Unix shell functionality**
  - **Example #2:** file redirection `` `wc p4.c > output.txt` ``
    - shell process calls calls `fork()`
    - child changes its output to write to `output.txt` instead of std output
    - child calls `exec()` to execute `wc p4.c`
    - `wc` is executed and results are written to `output.txt`
    - the shell process waits for the child using `wait()`

Process API: kill()

- `kill` - sends a signal to the process or a group of processes with specified pid(s)
  - `SIGTERM` - standard signal to terminate a process
  - SIGINT - interrupt from keyboard (Ctrl+C)
  - `SIGKILL` - force terminate immediately, all resources are released right away
  - SIGSTOP - process stops, program continues to hold its resources, doesn't use the CPU
  - **SIGKILL** and **SIGSTOP** cannot be caught, blocked or ignored
- Works via systems call `kill(pid, sig)` or by typing `kill <SIG> <PID>` in terminal
  - `kill -9 1234` will immediately terminate process 1234
  - -9 stands for SIGKILL
- run `man 3 signal` to see what signals are available on your terminal

Process API: ps

- `ps` - process listing - displays information about active processes
  - see the man page for the common flags
  - `ps aux` - displays information about all users (a, u), includes process not attached to terminal (x)
- Useful when need to locate a process id or inspect running processes

Process API: ps

cpu-api — -bash — 174×48

...esktop/lecture2_code — -bash | ...ocuments/cv_search — -bash | ~/Downloads — -bash | ...cuments/cv_search — -bash | ...de-master/cpu-api — -bash

```
Mac:cpu-api yulia$ ps aux
USER               PID  %CPU %MEM      VSZ    RSS   TT  STAT STARTED      TIME COMMAND
_windowserver      168   7.5  1.4 414956624 345648   ??  Ss   Sun04PM 222:46.90 /System/Library/PrivateFrameworks/SkyLight.framework/Resources/WindowServer -daemon
yulia              546   2.2  0.4 412622400  98368   ??  S    Sun04PM   0:50.59 /System/Applications/Utilities/Terminal.app/Contents/MacOS/Terminal
yulia              869   0.7  0.1 412282368  34368   ??  S    Sun04PM  45:30.57 /Applications/MenuMeters.app/Contents/MacOS/MenuMeters
_driverkit         368   0.6  0.2 426918368  38016   ??  Ss   Sun04PM  43:39.94 /System/Library/DriverExtensions/com.apple.DriverKit-AppleBCMWLAN.dext/com.apple.DriverKit-App
yulia            19050   0.6  0.2 412352752  55360   ??  S    Mon03PM  17:32.94 /Applications/TomatoBar.app/Contents/MacOS/TomatoBar
yulia            79168   0.3  0.3 427377312  79632   ??  Ss   11:44AM   0:10.46 /System/Library/ExtensionKit/Extensions/PaperKitExtension.appex/Contents/MacOS/PaperKitExtensi
root               161   0.2  0.1 426991280  25248   ??  Ss   Sun04PM  14:06.02 /usr/sbin/bluetoothd
yulia             3708   0.2  3.0 417892240 749168   ??  S    Sun06PM  40:34.39 /Applications/Keynote.app/Contents/MacOS/Keynote
yulia              569   0.1  0.0 411581280   9824   ??  S    Sun04PM   0:41.13 /System/Library/PrivateFrameworks/ViewBridge.framework/Versions/A/XPCServices/ViewBridgeAuxili
yulia              819   0.1  0.7 434417488 171328   ??  Ss   Sun04PM  28:17.79 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.GPU.xpc/Co
_mysql             311   0.1  0.1 35319720  17408   ??  Ss   Sun04PM  17:23.73 /usr/local/mysql/bin/mysqld --user=_mysql --basedir=/usr/local/mysql --datadir=/usr/local/mysq
yulia            79322   0.0  0.4 499135808  92320   ??  Ss   11:55AM   0:00.78 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.WebContent
yulia            79284   0.0  0.9 499355056 227536   ??  Ss   11:53AM   0:02.58 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.WebContent
yulia            79170   0.0  0.0 426916784   3456   ??  Ss   11:44AM   0:00.02 /System/Library/Frameworks/Metal.framework/Versions/A/XPCServices/MTLCompilerService.xpc/Conte
yulia            79169   0.0  0.1 426953712  13296   ??  Ss   11:44AM   0:00.07 /System/Library/Frameworks/Metal.framework/Versions/A/XPCServices/MTLCompilerService.xpc/Conte
yulia            79167   0.0  0.0 426947040   8784   ??  Ss   11:44AM   0:00.01 /System/Library/Frameworks/ExtensionFoundation.framework/Versions/A/XPCServices/extensionkitse
yulia            79136   0.0  1.2 499576112 310624   ??  Ss   11:42AM   0:02.33 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.WebContent
yulia            79096   0.0  0.9 499395504 215392   ??  Ss   11:39AM   0:05.54 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.WebContent
yulia            79082   0.0  1.2 499428016 295392   ??  Ss   11:38AM   0:03.58 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.WebContent
yulia            78251   0.0  0.1 426975280  15056   ??  Ss   10:38AM   0:01.10 /System/Library/PrivateFrameworks/IntelligencePlatformCompute.framework/Versions/A/XPCServices
yulia            78041   0.0  1.6 499722608 397472   ??  Ss   10:22AM   0:14.99 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.WebContent
yulia            77495   0.0  1.6 428739264 414992   ??  S     9:44AM   2:10.27 /System/Library/Services/AppleSpell.service/Contents/MacOS/AppleSpell
yulia            77433   0.0  0.6 499328784 157760   ??  Ss    9:39AM   0:12.60 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.WebContent
yulia            73443   0.0  0.1 426987616  28304   ??  S     4:16AM   0:01.37 /usr/libexec/remindd
root             67026   0.0  0.1 426974560  31024   ??  Ss    6:34PM   1:45.88 /usr/libexec/PerfPowerServices
yulia            66353   0.0  0.0 426962448   7232   ??  S     4:47PM   0:00.06 /System/Library/PrivateFrameworks/NewDeviceOutreach.framework/ndoagent
root             66248   0.0  0.0 426928928    768   ??  Ss    4:37PM   1:37.30 /usr/libexec/dirhelper
yulia            65796   0.0  0.0 410791616   3568 s004  S     4:13PM   0:00.10 -bash
root             65795   0.0  0.0 410744752   1808 s004  Ss    4:13PM   0:00.02 login -pfl yulia /bin/bash -c exec -la bash /bin/bash
root             63826   0.0  0.1 426967312  15856   ??  Ss    1:57PM   0:36.02 /System/Library/PrivateFrameworks/EcosystemAnalytics.framework/Support/ecosystemanalyticsd
yulia            63037   0.0  0.0 410321104   1824   ??  Ss    1:00PM   0:00.03 /System/Library/Frameworks/AudioToolbox.framework/XPCServices/com.apple.audio.SandboxHelper.xp
yulia            58433   0.0  0.1 411513888  19952   ??  Ss   Thu05AM   0:05.97 /System/Library/PrivateFrameworks/AppPredictionFoundation.framework/Versions/A/XPCServices/App
yulia            58432   0.0  0.2 427020640  46864   ??  S    Thu05AM   3:23.08 /usr/libexec/duetexpertd
yulia            57944   0.0  0.0 426929200   3072   ??  S    Thu04AM   0:00.05 /System/Library/CoreServices/Setup Assistant.app/Contents/Resources/mbuseragent
_datadetectors   57922   0.0  0.0 426932528   2192   ??  Ss   Thu04AM   0:00.04 /usr/libexec/DataDetectorsSourceAccess
yulia            54509   0.0  0.1 412131152  35632   ??  S    Wed10PM   0:03.63 /Library/Application Support/Microsoft/MAU2.0/Microsoft AutoUpdate.app/Contents/MacOS/Microsof
yulia            54498   0.0  0.2 412086976  45920   ??  S    Wed10PM   0:06.65 /Library/Application Support/Microsoft/MAU2.0/Microsoft AutoUpdate.app/Contents/MacOS/Microsof
yulia            52741   0.0  0.1 412015344  27392   ??  S    Wed07PM   0:06.17 /System/Library/CoreServices/Family.app/Contents/MacOS/Family
yulia            51211   0.0  0.0 410349776   1824   ??  Ss   Wed06PM   0:00.03 /System/Library/Frameworks/AudioToolbox.framework/XPCServices/com.apple.audio.SandboxHelper.xp
yulia            51207   0.0  0.1 431566176  16800   ??  Ss   Wed06PM   0:02.93 /System/Library/Frameworks/WebKit.framework/Versions/A/XPCServices/com.apple.WebKit.GPU.xpc/Co
yulia            50300   0.0  0.1 431547920  28528   ??  Ss   Wed04PM   0:01.20 /System/Library/PrivateFrameworks/UIFoundation.framework/Versions/A/XPCServices/nsattributedst
yulia            50299   0.0  0.1 412048624  13360   ??  Ss   Wed04PM   0:00.83 /Applications/Dropbox.app/Contents/XPCServices/DropboxActivityProvider.xpc/Contents/MacOS/Drop
yulia            50293   0.0  0.1 454087984  33472   ??  S    Wed04PM   0:00.45 /Applications/Dropbox.app/Contents/Frameworks/Dropbox Helper (Renderer).app/Contents/MacOS/Dro
root             50285   0.0  0.0 410974080   8416   ??  S    Wed04PM   0:09.05 /Library/DropboxHelperTools/Dropbox_u501/dbkextd
yulia            50275   0.0  0.1 453555632  35136   ??  S    Wed04PM   0:00.59 /Applications/Dropbox.app/Contents/Frameworks/Dropbox Helper (Renderer).app/Contents/MacOS/Dro
yulia            50274   0.0  0.4 454353472  93984   ??  S    Wed04PM   1:25.28 /Applications/Dropbox.app/Contents/Frameworks/Dropbox Helper (Renderer).app/Contents/MacOS/Dro
```

Process API: ps

user running the process

process id

cpu and physical memory usage

resident set size (RSS) - memory used by the process and any shared libraries it invoked

total CPU time

command

```
Mac:cpu-api yulia$ ps aux
USER              PID  %CPU %MEM      VSZ    RSS   TT  STAT STARTED      TIME COMMAND
_windowserver     168   7.5  1.4 414956624 345648   ??  Ss   Sun04PM 222:46.90 /System/Library/Priv
yulia             546   2.2  0.4 412622400  98368   ??  S    Sun04PM   0:50.59 /System/Applications
yulia             869   0.7  0.1 412282368  34368   ??  S    Sun04PM  45:30.57 /Applications/MenuMe
_driverkit        368   0.6  0.2 426918368  38016   ??  Ss   Sun04PM  43:39.94 /System/Library/Driv
yulia           19050   0.6  0.2 412352752  55360   ??  S    Mon03PM  17:32.94 /Applications/Tomato
```

virtual memory size (VSZ) - amount of memory available to the process (not necessarily used)

which terminal

?? not started at a terminal

process status

S - sleeping (waiting for something)

D - performing I/O

R - running or on the run queue

T - stopped

Z - zombie (terminated but not reclaimed by the parent)

s - session leader (shell or its child)

Process API: ps

- Can be combined with other commands
  - `ps aux | grep python3` - will pipe output of ps to grep, which will grab only python3 processes

Process API: top

- `top` - shows the same information as `ps`
  - continuously updated
  - interactive

Process API: htop

```
  0[|||                          6.6%]  3[                          0.0%]  5[                          0.0%]  8[                          0.0%]
  1[||||                         7.9%]  4[|                         0.7%]  6[                          0.0%]  9[                          0.0%]
  2[||                           3.3%]                                     7[|                         0.7%]
Mem[||||||||||||||||||||                                   3.25G/24.0G] Tasks: 773, 1974 thr, 0 kthr; 2 running
Swp[||||||||||||||||||||
```

Summary

- Processes are **running programs** managed by an OS
- Each process has a **state**: address space (memory), CPU registers (PC, stack pointer, etc), and I/O status (open files, etc.)
- OS has **data structures** (PCB) and **system calls** to manage state of the processes
- `fork()`, `exec()`, `wait()` and `kill()` are the **key system calls** that OS exposes to create, modify, suspend, and terminate processes
- `ps, top, htop` are **useful utilities** for monitoring processes

Reading

- Chapter 4 The abstraction: The Process
  - https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf
- Chapter 5 Interlude: Process API
  - https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf
- Code from today's lecture
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/cpu-api
- Optional reading
  - Chapter 3.1, 3.3 Processes, Siberchatz

Quiz 2

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet

Please keep the noise down

- If you get disconnected and cannot use the same id, append an underscore (_) before the id, e.g. _9999999